02624 Inverse Problems and Imaging — Homework 1

Jeppe Klitgaard <s250250@dtu.dk>
Date: 2025-10-24

Contents

Introduction 0
1) Backwards Heat Equation 1
  1.a) Singular System 1
  1.b) Discretisation of K 1
  1.c) Inversion using Truncated SVD 2
2) Abel Problem 3
  2.*) Proving Compactness of K 4
  2.a) Computing the Forward Model and Noisy Measurements 4
  2.b) Truncated SVD Reconstruction 4
    2.b.i) Reconstruction Bias at Boundary $s = 1$ 5
  2.c) Tikhonov Regularisation 5
  2.d) Regularisation of a Discontinuous Function 6
3) Blurring Problem 7
  3.a) Showing that $\tilde{R}_\alpha$ is a Regularisation Operator 7
  3.b) Convergence in Deblurring Problem 8
A) Appendix 9
  A.1) Orthogonality of Sine Functions 9
  A.2) Optional Material for Section 2 9
  A.3) Large Figures for Section 1 10
  A.4) Large Figures for Section 2 12
  A.5) Large Figures for Section 3 15
  A.6) Code for Section 1.b 16
  A.7) Code for Section 1.c 18
  A.8) Code for Section 2.a 19
  A.9) Code for Section 2.b 20
  A.10) Code for Section 2.c 23
  A.11) Code for Section 2.d 24
  A.12) Code for Section 3.b 25
Bibliography 29

Introduction

This report presents solutions to Homework 1 for the course 02624 Inverse Problems and Imaging.

As discussed in class with Professor Kim Knudsen, the report spans slightly more than the initially required 7 pages in order to improve readability.

Code and larger versions of the figures can be found in the appendix alongside optional tasks and lengthier intermediate mathematical derivations.

1) Backwards Heat Equation

We are given the *forward model* of the *backwards heat equation*:

$$Kx(t) = \int_0^\pi x(s)k(t,s)\,\mathrm{d}s \quad \text{where} \quad k(t,s) = \frac{2}{\pi}\sum_{n=1}^{\infty} e^{-Tn^2}\sin(nt)\sin(ns) \tag{1}$$

1.a) Singular System

Using that K is compact in $L^2(0,\pi)$, we can demonstrate that it has the singular system $\left(x_j, y_j, \mu_j\right)$ where the *singular functions*, x_j, y_j are given by:

$$x_j(t) = \sqrt{\frac{2}{\pi}}\sin(jt) \qquad y_j(s) = \sqrt{\frac{2}{\pi}}\sin(js) \tag{2}$$

And the singular values, μ_j are given by:

$$\mu_j = e^{-Tj^2} \tag{3}$$

To verify that this is indeed the singular system for K, we recall the definition of a singular system [1, Theorem A.57], for which it holds for every $j \in J \subseteq \mathbb{N}_1$:

$$Kx_j = \mu_j y_j \qquad K^* y_j = \mu_j x_j \tag{4}$$

We first verify that $Kx_j = \mu_j y_j$:

$$\begin{aligned} Kx_j(t) &= \int_0^\pi x_j(s)k(t,s)\,\mathrm{d}s = \int_0^\pi \sqrt{\frac{2}{\pi}}\sin(js)\left(\frac{2}{\pi}\sum_{n=1}^{\infty} e^{-Tn^2}\sin(nt)\sin(ns)\right)\mathrm{d}s \\ &= \left(\frac{2}{\pi}\right)^{\frac{3}{2}}\left(\sum_{n=1}^{\infty} e^{-Tn^2}\sin(nt)\int_0^\pi \sin(js)\sin(ns)\,\mathrm{d}s\right) = \left(\frac{2}{\pi}\right)^{\frac{3}{2}}\left(\sum_{n=1}^{\infty} e^{-Tn^2}\sin(nt)\frac{\pi}{2}\delta_{jn}\right) \\ &= e^{-Tj^2}\sqrt{\frac{2}{\pi}}\sin(jt) = \mu_j y_j(t) \end{aligned} \tag{5}$$

Where the summation and integral interchange was enabled by uniform convergence of the series through inspection of (1), in which the Gaussian decay factor e^{-Tn^2} ensures rapid convergence. The orthogonality of the sine function in $L^2(0,\pi)$ is shown in Appendix Section A.1, where we find that the integral over the sines of different integer frequencies n, j is given by $\frac{\pi}{2}\delta_{jn}$ as shown in (A.43).

To consider the other equation in (4), we must first determine the adjoint operator, K^*, which is obtained through the conjugate-transpose of the kernel.

$$k^*(s,t) = k(\bar{t}, s) \tag{6}$$

By noting that $k(s,t)$ is real-valued and symmetric by inspection of (1), we have:

$$k^*(s,t) = k(t,s) \tag{7}$$

Such that:

$$K^* y(s) = \int_0^\pi y(t)k^*(s,t)\,\mathrm{d}t = \int_0^\pi y(t)k(t,s)\,\mathrm{d}t = Ky(s) \tag{8}$$

From which we may conclude that K is *self-adjoint*, $K = K^*$.

As such, we may repeat treatment above for the equation $K^* y_j = \mu_j x_j$, which becomes true by inspection.

In addition to this, the singular functions must be orthonormal in $L^2(0,\pi)$, which they are by the treatment in Section A.1 and the singular values must be non-negative, which is indeed the case for the given system.

Lastly, we note that the singular values decay to zero as $j \to \infty$ since $T > 0$, as we would expect for a compact operator.

We conclude that K is indeed a compact operator with singular system $\left(x_j, y_j, \mu_j\right)$ as given in (2) and (3).

1.b) Discretisation of K

We seek to discretise K by a matrix $A \in \mathbb{R}^{n\times n}$ where $n = 128$ and $T = 0.03$, to which end the spatial and temporal domains $[0,\pi]$ are discretised into n equidistant points:

$$t_i = \frac{k\pi}{n+1} \qquad s_i = \frac{k\pi}{n+1} \qquad \text{where} \qquad k = 1, \ldots, n \tag{9}$$

Where we note that the ends of the interval are not included as the sine functions are zero at these points, which would not result in an orthogonal basis.

Next, we construct a matrices U, V whose columns are the left and right singular vectors, x_j, y_j, sampled at the points s_i:

$$U_{ij} = \sqrt{\frac{2}{n+1}} \sin(j \cdot t_i) \qquad V_{ij} = \sqrt{\frac{2}{n+1}} \sin(j \cdot s_i) \tag{10}$$

Where the scaling factor ensures that the columns are normalised.

From this, we may construct the diagonal matrix Σ whose entries are the singular values μ_j:

$$\Sigma_{jj} = \mu_j = e^{-Tj^2} \tag{11}$$

Which enables us to construct the discretisation of K directly:

$$A = U\Sigma V^{\mathsf{T}} \tag{12}$$

Here, we have exploited that we have analytical expressions for the components of the singular system. In a more general setting, we may have to resort to approximation of the integrals in (1) using a quadrature rule.

We can compare the singular values of A against those for K obtained by analytical means, as seen in Figure 1. We observe that they decay rapidly at the same rate up until the 33rd singular value, after which the singular values of A remain constant at a value determined by the numerical precision of the computation, which is approximately 10^{-16} in this case, as marked by the dotted black line.

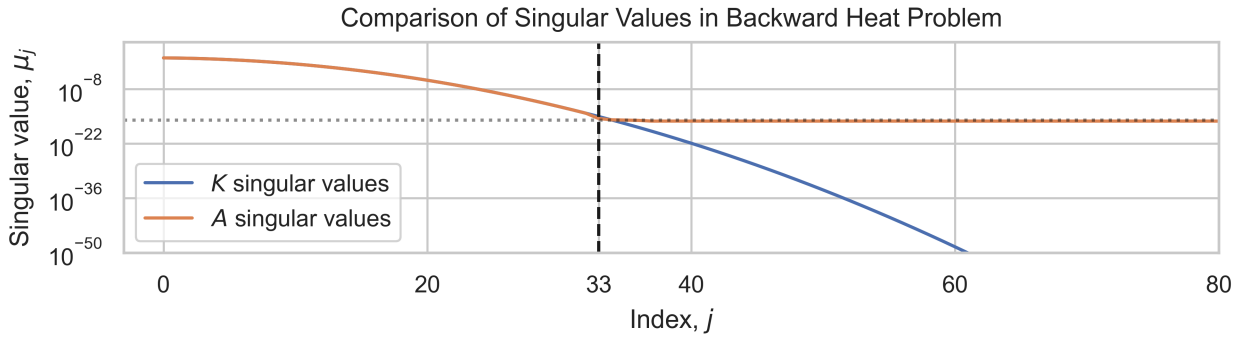


Figure 1: Singular values of the discretised operator A for $n = 128$ and $T = 0.03$ compared against the analytical expression for the singular values of K.

Similarly, we can compare the singular vectors of A against the singular functions of K, as seen in Figure 2. We observe an excellent match between the discretised vectors and the continuous singular functions at lower *modes*. At higher frequency modes, namely those past the $j = 33$ threshold, we observe that the limited machine precision affects the accuracy of the representation and noise dominates the decomposition.

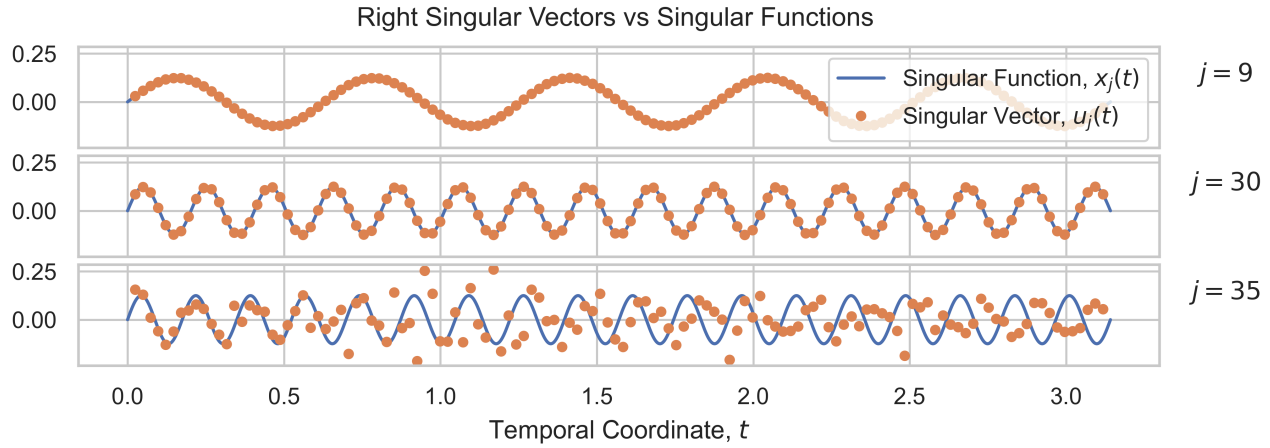


Figure 2: 3 select singular vectors and their associated singular functions for $n = 128$ and $T = 0.03$.

1.c) Inversion using Truncated SVD

Given the function:

$$x(s) = s \sin(s) \exp((s - \pi/2)^2) \tag{13}$$

We seek to recover a solution to the inverse problem $Kx = y$ given noisy measurements y^δ. We construct our measurements by computing the forward model $y = Kx$ and adding Gaussian noise with relative size $\|n\|_{L^2(0,\pi)} / \|y\|_{L^2(0,\pi)} = 2\%$.

A naïve inversion of the problem using the inverse of A is expected to perform poorly due to the hegemony of noise at higher frequency modes. We may instead use a *truncated SVD* approach in which only the first $j^\star$ singular values are included in the inversion.

Finding a suitable choice of the threshold index $j^\star$ is undertaken through the construction of a *Picard plot* containing the singular values μ_j, the expansion coefficients $\left|\left(y^\delta, y_j\right)\right|$ and the solution coefficients $\left|\left(y^\delta, y_j\right)\right|/\mu_j$ as seen in Figure 3.

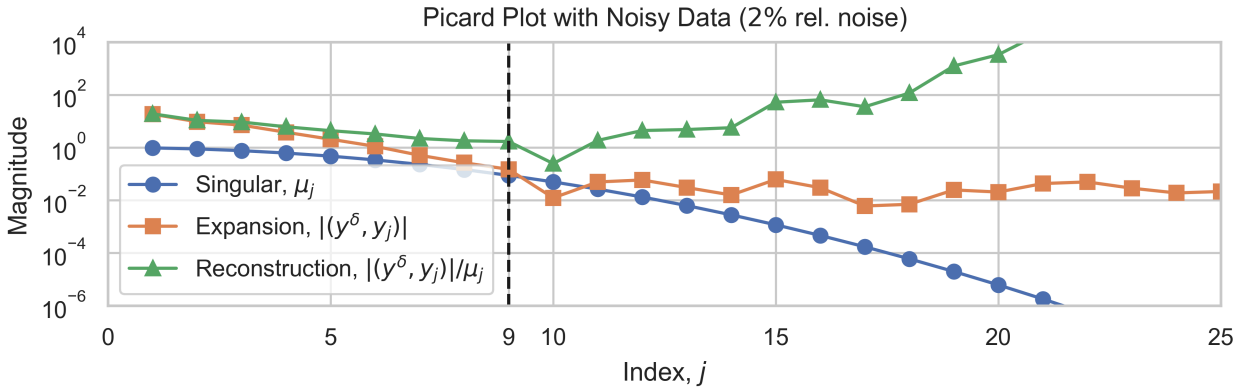


Figure 3: Picard plot of the noisy data y^δ with 2% relative noise.

We recall the *discrete Picard condition* [2, 3.16], which states that the expansion coefficients must decay faster than the singular values for a stable solution to exist. We use this in conjunction with Figure 3 to choose a threshold index $j^\star = 9$ at which the solution coefficients begin to diverge, indicating that noise is dominating the solution. We note that this choice is somewhat subjective and given we only have a single realisation of the noise available in the plot, we conservatively place it at $j^\star = 9$ as opposed to $j^\star = 10$.

With this, we are able to compute a reconstruction of x using the truncated SVD:

$$U\Sigma V^\dagger = A \tag{14}$$

From which we may construct the truncated matrices by reducing the number of columns of U, V and the size of Σ to $j^\star$.

From this, we may reconstruct x using (4):

$$x_{\text{rec}} = \sum_{j=1}^{j^\star} \frac{\left(y^\delta, y_j\right)}{\mu_j} x_j = \underbrace{\tilde{V}\tilde{\Sigma}^{-1}\tilde{U}^\dagger}_{\tilde{A}^{-1}} y^\delta \tag{15}$$

Where the tildes denote that the matrices are truncated to the first $j^\star$ columns/rows.

The resulting reconstruction is shown in Figure 4, where we observe a good match between the true model x and the reconstruction x_{rec}. The noisy measurement y^δ is also shown alongside the true signal y.

We observe that the regularised inversion is able to recover the main features of x despite the presence of significant noise in the measurements. Attempting to invert the problem without regularisation results in a completely unusable solution dominated by noise, as seen by x_{naive} in Figure 4, whose values are to be read off the right vertical axis.

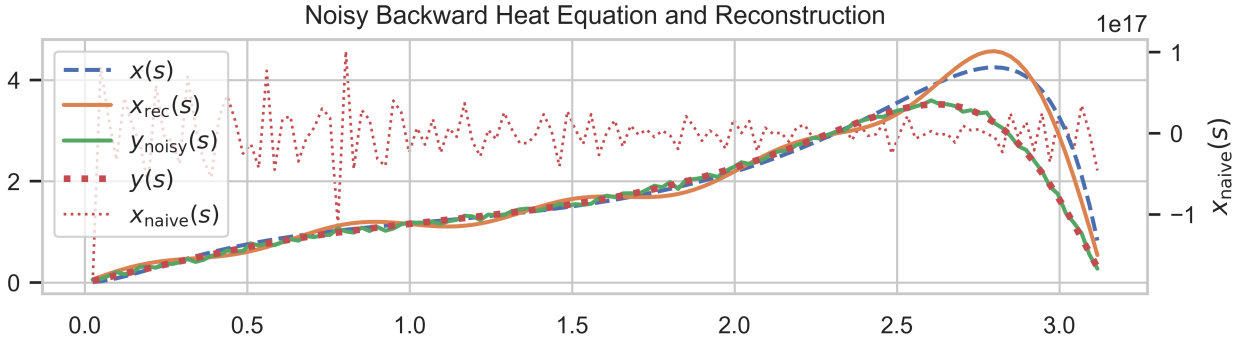


Figure 4: Reconstruction of x using truncated SVD with $j^\star = 9$ compared against the true x.

2) Abel Problem

We are given the *forward model* of the *Abel problem* on the domain $L^2(0, 1)$:

$$Kx(t) = \int_0^t \frac{1}{\sqrt{t-s}} x(s)\, \mathrm{d}s \tag{16}$$

2.*) Proving Compactness of K

Due to space constraints, this section has been relegated to the appendix in Section A.2

2.a) Computing the Forward Model and Noisy Measurements

We seek to compute the forward model $y = Kx$ associated with the Abel problem given in (16) for the function:

$$x(s) = s^2 \tag{17}$$

We construct noisy measurements by addition of Gaussian noise with relative size $\|n\|_{L^2(0,1)}/\|y\|_{L^2(0,1)} = 5\%$ such that:

$$y^\delta = y + n \tag{18}$$

Which produces the realisation seen in Figure 5.

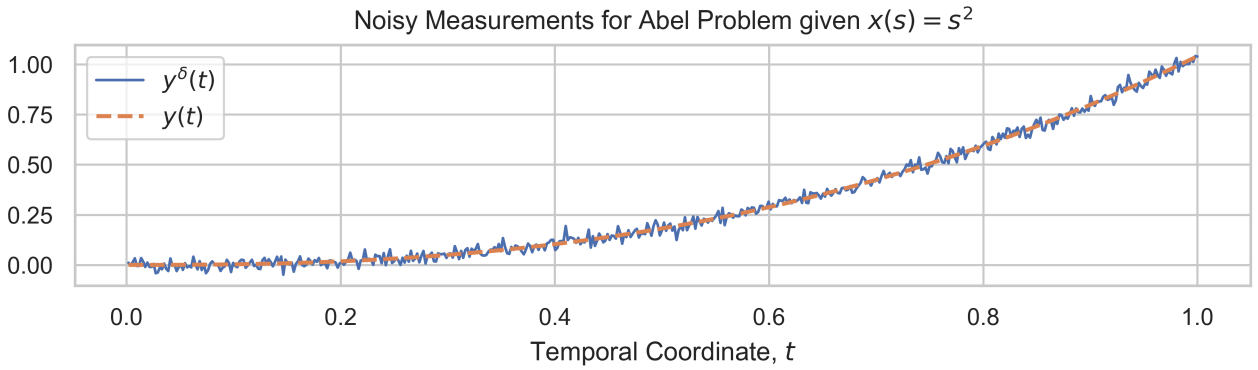


Figure 5: Noisy realisation of Abel problem with $x(s) = s^2$ perturbed by 5% relative white noise.

The code for this section can be found in Section A.8.

2.b) Truncated SVD Reconstruction

In similar fashion to Section 1.c, we seek to reconstruct x from the noisy measurements y^δ using a truncated SVD approach already described in Section 1.c.

As in Section 1.c, we create a Picard plot as seen in Figure 6 in order to gauge the information carried by the components of the SVD.

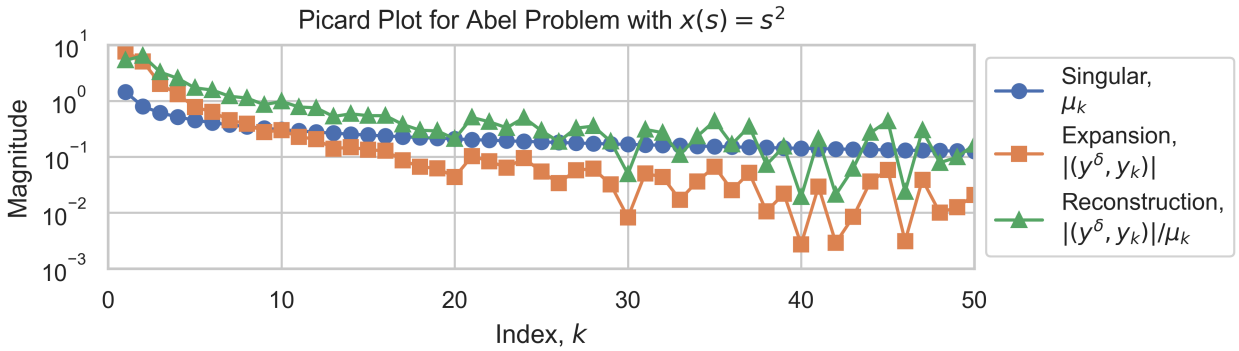


Figure 6: Picard plot of the noisy data y^δ with 5% relative noise.

Qualitatively, we observe that components past $k \approx 20$ are dominated by the noise, which helps guide our selection of truncation levels k to consider for our reconstructions seen in Figure 7.

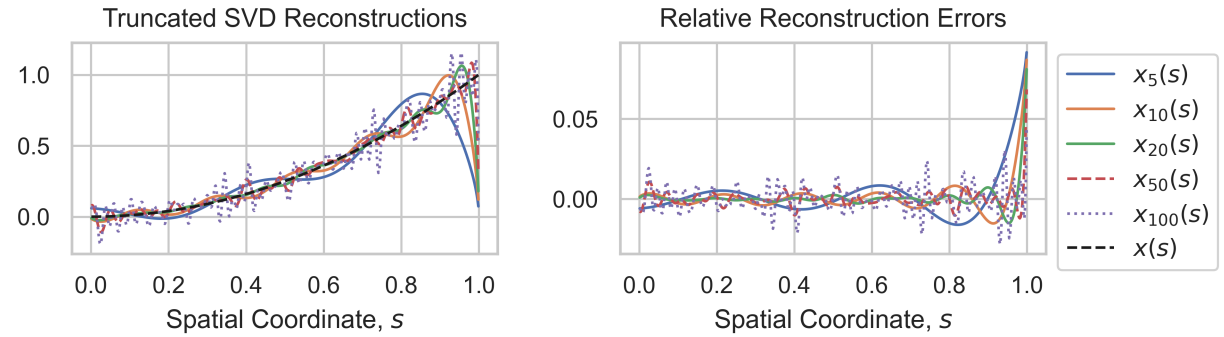


Figure 7: Reconstructions of $x(t)$ using truncated SVD with varying truncation levels k compared against the true $x(t)$. Second panel shows the relative error given by $(x - x_k)/\|x\|_2$.

Figures 6 and 7 both indicate that the problem under consideration is much less ill-posed than that of the backwards heat equation in Section 1, as we are able to perform a stable, albeit noisy, inversion even at high truncation levels such as $k = 100$.

Nevertheless, the analysis of the Picard plot is in agreement with the results of Figure 7, which both indicate that truncation levels around $k = 20$ provide a good balance between variance and bias in the reconstruction.

We notice by inspection of the relative errors in the second panel of Figure 7 that a significant bias is introduced at the boundary $s = 1$.

2.b.i) Reconstruction Bias at Boundary $s = 1$

Having observed that the TVSD reconstructions drop to zero at $s = 1$ for all truncation levels, we investigate further by considering the Truncated SVD reconstruction as given in (15):

$$x_{\text{rec}}(s) = \sum_{j=1}^{k} \frac{\left(y^{\delta}, y_j\right)}{\mu_j} x_j(s) \tag{19}$$

Since neither $\left(y^{\delta}, y_j\right)$ nor μ_j depend on s nor are generally zero, we focus on the singular functions $x_j(s)$, which are related to the forward operator K through the adjoint operator K^* as per (4):

$$\left(K^*Kx_j\right)(s) = \mu_j^2 x_j(s) \tag{20}$$

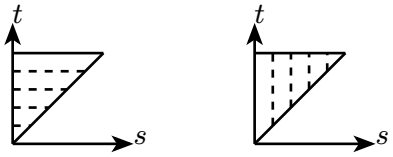
Which requires us to find an expression for the adjoint operator K^*, defined through the inner product:

$$\begin{aligned}(Kx, y) = (x, K^*y) &= \int_0^1 \left(\int_0^t \frac{1}{\sqrt{t-s}} x(s)\,\mathrm{d}s \right) \overline{y(t)}\,\mathrm{d}t \\ &= \int_0^1 x(s) \left(\int_s^1 \frac{1}{\sqrt{t-s}} \overline{y(t)}\,\mathrm{d}t \right) \mathrm{d}s = (x, K^*y)\end{aligned} \tag{21}$$

Where the interchange of the integrals is enabled by Fubini's theorem and we identify the adjoint operator as:

$$K^*y(s) = \int_s^1 \frac{1}{\sqrt{t-s}} y(t)\,\mathrm{d}t \tag{22}$$

The transformation of the limits may be appreciated by considering the sketch below:



This allows us to reconsider the left singular functions $x_j(s)$ at $s = 1$:

$$\left(K^*Kx_j\right)(s=1) = \int_{s=1}^1 \int_0^t \frac{1}{\sqrt{t-\tau}} y(\tau)\,\mathrm{d}\tau = 0 = \mu_j^2 x_j(s=1) \tag{23}$$

Where the outer integrals upper and lower limits are equal, resulting in a zero value for any reconstruction evaluated at $s = 1$. Given $\mu_j \neq 0$, we find that $x_j(s=1) = 0$ for all j.

Returning to the reconstruction expression of (19), we find that:

$$x_{\text{rec}}(s=1) = \sum_{j=1}^{k} \frac{\left(y^{\delta}, y_j\right)}{\mu_j} x_j(s=1) = 0 \tag{24}$$

We note that the proof does not immediately extend to the discrete case where the outer integral is approximated by a quadrature rule, though given the discretised operator must approximate its continuous equivalent with increasingly fine discretisations, it must hold $x_{\text{rec}}(s=1)$ is biased towards zero as the discretisation is refined.

2.c) Tikhonov Regularisation

We additionally consider Tikhonov regularisation parameterised by α to produce reconstructions x_α given noisy measurements y^{δ}. There are different equivalent formulations of Tikhonov regularisation; we consider the following closed-form formulation:

$$\begin{aligned}x_\alpha = \tilde{R}_\alpha y^{\delta} &= \left(K^*K + \alpha I\right)^{-1} K^* y^{\delta} \\ &= \left(A^{\mathsf{T}}A + \alpha I\right)^{-1} A^{\mathsf{T}} y^{\delta}\end{aligned} \tag{25}$$

Which is easily computed for a suitable range of parameters $\alpha \in [10^{-4}, 10]$ in order to obtain the reconstructions as seen in Figure 9.

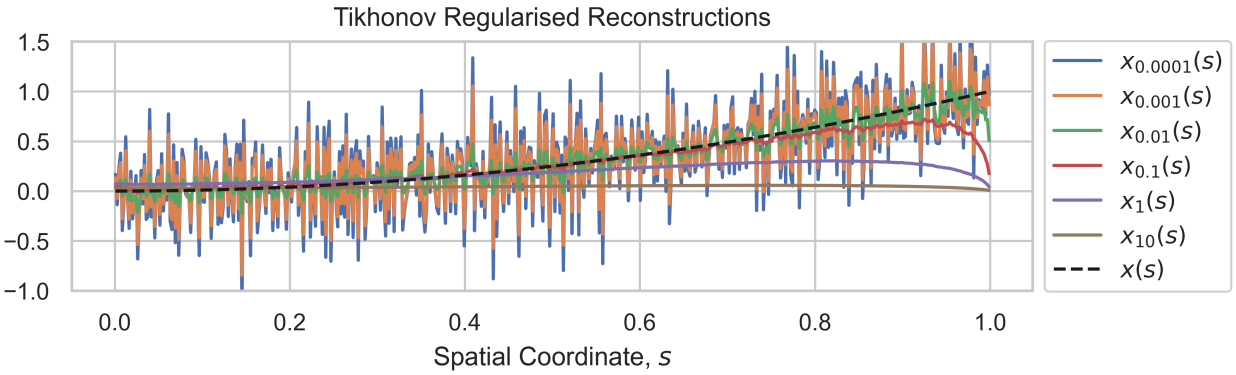


Figure 9: Reconstructions of $x(t)$ using Tikhonov regularisation with varying regularisation parameters α compared against the true $x(t)$.

We observe that $\alpha \approx 10^{-2}$ appears to be a good balance between noise suppression and the bias introduced by regularisation which drives the reconstruction towards zero at $s = 1$ as also observed for the TSVD regularisation in Section 2.b.

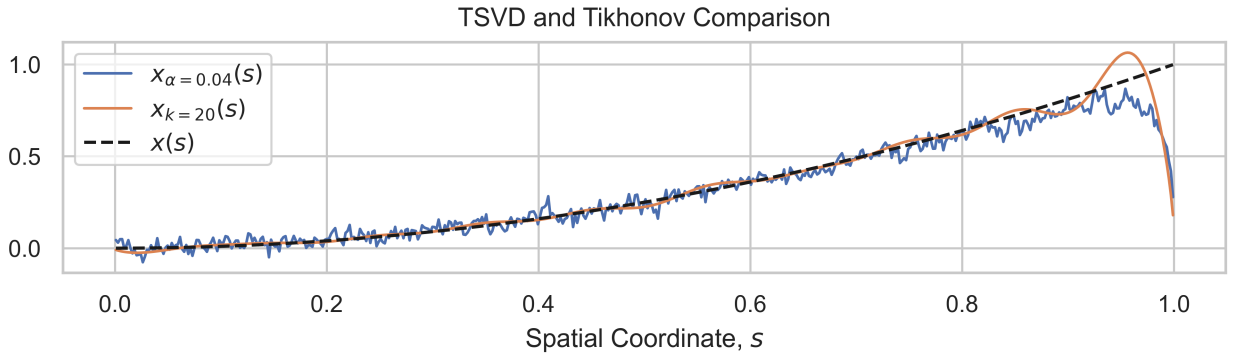


Figure 10: Comparison of TSVD reconstruction with $k = 20$ and Tikhonov reconstruction with $\alpha = 4 \times 10^{-2}$.

By comparing the two types of reconstruction in Figure 10, we observe that they have rather different characteristics. The Tikhonov reconstruction is not subject to the systematic bias introduced by the crude truncation of the higher modes in the TSVD approach, but also does not suppress the high-frequency noise as effectively as observed in the TSVD reconstruction. As such, the choice between the two methods will depend on the application at hand and poses the familiar bias-variance trade-off associated with regularisation.

2.d) Regularisation of a Discontinuous Function

We attempt the above regularisation methods on a discontinuous function constructed from the original quadratic function and a scaled Heaviside step function:

$$x(s) = s^2 + 4 \cdot H\left(s - \frac{1}{2}\right) \tag{26}$$

Where $H(x)$ is the Heaviside step function defined as:

$$H(x) = \begin{cases} 0 & \text{if} \quad x < 0 \\ \frac{1}{2} & \text{if} \quad x = 0 \\ 1 & \text{if} \quad x > 0 \end{cases} \tag{27}$$

Which allows us to produce another comparison of the two regularisation methods as seen in Figure 11.

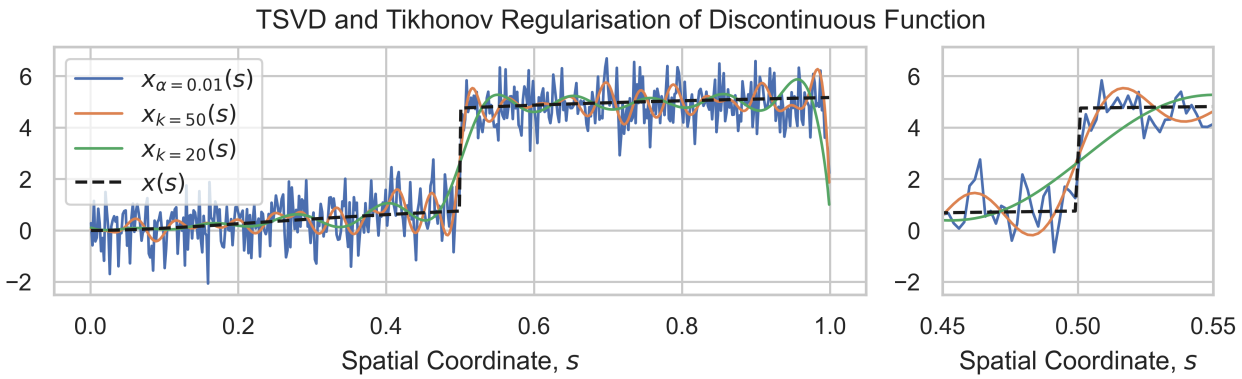


Figure 11: Comparison of TSVD reconstructions with $k \in \{20, 50\}$ and Tikhonov reconstruction with $\alpha = 10^{-2}$ for a discontinuous function.

We observe that the regularisation methods struggle to accurately reconstruct the discontinuity at $s = \frac{1}{2}$, which is to be expected given that both methods implicitly assume some degree of smoothness in the underlying function being reconstructed. Increasing k or decreasing α allows for better reconstruction of the high-frequency components associated with the discontinuity at the cost of decreased noise suppression.

3) Blurring Problem

We are given an operator $\tilde{R}_\alpha$ defined as:

$$x_a = \tilde{R}_\alpha y = \sum_j \frac{\mu_j^2}{\mu_j^3 + \alpha}(y, y_j)x_j \tag{28}$$

3.a) Showing that $\tilde{R}_\alpha$ is a Regularisation Operator

We seek to show that:

$$\lim_{\alpha \to 0} \tilde{R}_\alpha K x = x \tag{29}$$

By injectivity the singular functions x_j, y_j span their respective spaces such that any $x \in X$ may be decomposed as:

$$x = \sum_j (x, x_j)x_j \tag{30}$$

Similarly, using $Kx_j = \mu_j y_j$ we decompose y as:

$$y = Kx = \sum_j \mu_j (x, x_j)y_j \tag{31}$$

Which allows us to consider the inner product (y, y_j):

$$\begin{aligned}(y, y_j) = (Kx, y_j) = \left(\sum_k \mu_k (x, x_k)y_k, y_j\right) = \sum_k \left[\mu_k (x, x_k)(y_k, y_j)\right] = \sum_k \left[\mu_k (x, x_k)\delta_{kj}\right] \\ = \mu_j (x, x_j)\end{aligned} \tag{32}$$

Where the summation and inner product interchange by linearity of the inner product and $(y_k, y_j) = \delta_{kj}$ by orthonormality of the basis functions.

Now we consider the norm of the difference between $\tilde{R}_\alpha K x$ and x:

$$\left\|\tilde{R}_\alpha K x - x\right\|^2 = \left\|\sum_j \left[\frac{\mu_j^2}{\mu_j^3 + \alpha}(y, y_j)x_j\right] - \sum_j (x, x_j)x_j\right\|^2 = \left\|\sum_j \left[\left(\frac{\mu_j^3}{\mu_j^3 + \alpha} - 1\right)(x, x_j)x_j\right]\right\|^2 \tag{33}$$

Onto which we leverage that $\{x_j\}$ is complete by injectivity and orthonormal in order to apply Plancherel theorem / Parseval's Identity [1, Theorem A.15]:

$$\sum_{k=1}^{\infty} |(x, x_k)_X|^2 = \|x\|_X^2 \tag{34}$$

In order to recover:

$$\left\|\tilde{R}_\alpha K x - x\right\|^2 = \sum_j \left[\left|\frac{\mu_j^3}{\mu_j^3 + \alpha} - 1\right|^2 \left|(x, x_j)\right|^2\right] \tag{35}$$

Of which we take the limit $\alpha \to 0^+$:

$$\lim_{\alpha\to 0^+} \left\|\tilde{R}_\alpha K x - x\right\|^2 = \lim_{\alpha\to 0^+} \sum_j \left[\left|\frac{\mu_j^3}{\mu_j^3+\alpha} - 1\right|^2 \left|(x, x_j)\right|^2\right] = \sum_j \left[0 \cdot \left|(x, x_j)\right|^2\right] = 0 \tag{36}$$

Where the interchange of the sum and limit are justified by satisfying the conditions of *Lebesque's Dominated Convergence Theorem*, which requires *pointwise convergence* and *domination by an integrable function*:

$$\begin{aligned} &\text{Pointwise Convergence:} && \lim_{\alpha\to 0^+} \left|\frac{\mu_j^3}{\mu_j^3+\alpha} - 1\right|^2 \left|(x, x_j)\right|^2 = 0 && \forall \quad j \\ &\text{Domination:} && \left|\frac{\mu_j^3}{\mu_j^3+\alpha} - 1\right|^2 \left|(x, x_j)\right|^2 \leq \left|(x, x_j)\right|^2 && \forall \quad j, \alpha > 0 \end{aligned} \tag{37}$$

As such, given that the norm of their difference goes to zero as $\alpha \to 0^+$, we conclude that $\lim_{\alpha\to 0^+} \left\|\tilde{R}_\alpha K x - x\right\| = 0$.

3.b) Convergence in Deblurring Problem

We consider the blurring problem:

$$Kx(t) = \int_{-6}^{6} k(t-s)x(s)\,\mathrm{d}s \qquad \text{where} \qquad k(z) = \begin{cases} 1+\cos(\pi z/3) & \text{if} \quad |z| < 3 \\ 0 & \text{if} \quad |z| \geq 3 \end{cases} \tag{38}$$

We select a test function $x(s)$ given by the sum of a Gaussian and a *top hat* function:

$$x(s) = \exp(-2(s+2)^2) + H(s-1) - H(s-3) \tag{39}$$

And introduce measurement noise according to parameter δ such that $y^\delta = y + d$ where $\|d\| < \delta$ and construct regularisation parameter $\alpha = \delta^3$ using the regularisation technique described in Section 3.

In order to validate our implementation we construct a realisation for $\delta = 10^{-7}$ as seen in Figure 12.

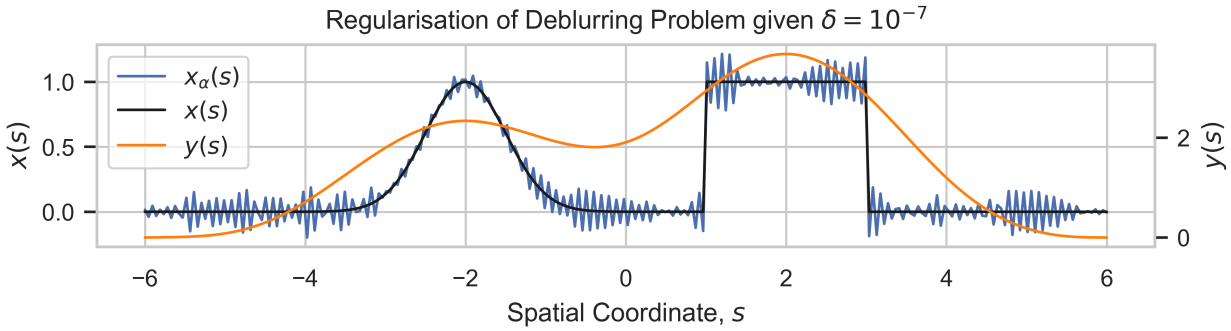


Figure 12: Validation of deblurring implementation with $\delta = 10^{-7}$ and $\alpha = \delta^3$.

Satisfied that our implementation is correct, we proceed to investigate the convergence of the the regularisation regime by considering $\|x_\alpha - x\|_2$ where $x_a = \tilde{R}_{\alpha(\delta)} y^\delta$ over a range of noise levels $\delta \in [1, 10^{-7}]$ as seen in Figure 13.

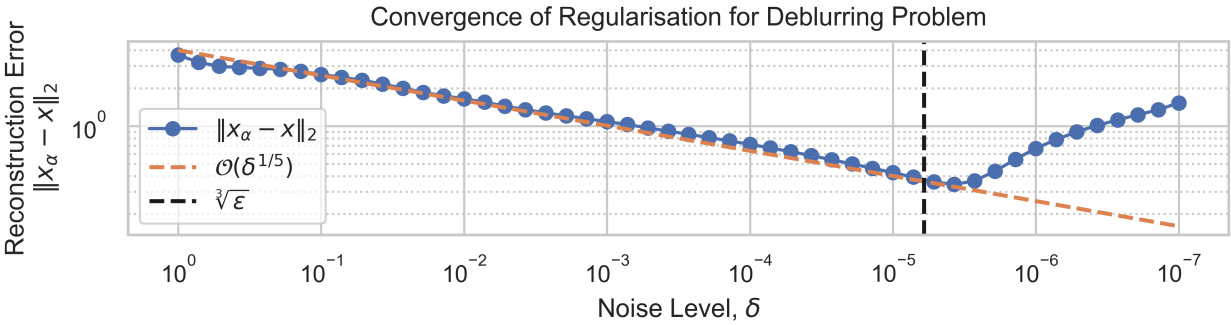


Figure 13: Convergence of regularised solution x_α to true solution x as noise level $\delta \to 0$ with $\alpha = \delta^3$.

We notice that the convergence rate appears to be of order $\mathcal{O}\left(\delta^{1/5}\right)$ and that the convergence in the two norm breaks at $\delta \approx 5 \times 10^{-6}$, which corresponds to $\alpha = \delta^3$ reaching the numerical precision of the computation, $\varepsilon \approx 2.22 \times 10^{-16}$.

As such, we were able to numerically demonstrate convergence of the regularised solution to the true solution with decreasing noise levels.

A) Appendix

A.1) Orthogonality of Sine Functions

We wish to show that the sine functions are orthogonal in $L^2(0,\pi)$:

$$\begin{aligned}\int_0^\pi \sin(js)\sin(ns)\,\mathrm{d}s &= \int_0^\pi \frac{1}{2}[\cos((n-j)s) - \cos((n+j)s)] \\ &= \frac{1}{2}\left[\frac{\sin((n-j)s)}{n-j} - \frac{\sin((n+j)s)}{n+j}\right]_0^\pi \\ &= \frac{1}{2}\left[\frac{\sin((n-j)\pi)}{n-j} - \frac{\sin((n+j)\pi)}{n+j}\right]\end{aligned} \tag{A.40}$$

Where $n, j \in \mathbb{N}_1$

We now consider the case $j \neq n$ such that $n - j = p \in \mathbb{Z}$ and $n + j = q \in \mathbb{Z}$:

$$\int_0^\pi \sin(js)\sin(ns)\,\mathrm{d}s = \frac{1}{2}\left[\frac{\sin(p\pi)}{p} - \frac{\sin(q\pi)}{q}\right] = \frac{1}{2}[0-0] = 0 \tag{A.41}$$

The other case, $j = n$ is more interesting and gives:

$$\begin{aligned}\int_0^\pi \sin(js)\sin(ns)\,\mathrm{d}s &= \frac{1}{2}\left[\frac{\sin((n-j)\pi)}{n-j} - \frac{\sin((2n)\pi)}{2n}\right] \\ &= \frac{1}{2}\frac{\sin((n-j)\pi)}{n-j} \\ &= \frac{1}{2}\frac{1}{n-j}\left[(n-j)\pi - \frac{(n-j)^3\pi^3}{3!} + \frac{(n-j)^5\pi^5}{5!} - \ldots\right] \\ &= \frac{1}{2}\left[\pi - \frac{(n-j)^2\pi^3}{3!} + \frac{(n-j)^4\pi^5}{5!} - \ldots\right] \\ &= \frac{\pi}{2} + 0\end{aligned} \tag{A.42}$$

Where we have used the power series definition of the sine function.

As such, we find that the integral in (A.40) evaluates to:

$$\int_0^\pi \sin(js)\sin(ns)\,\mathrm{d}s = \frac{\pi}{2}\delta_{jn} \tag{A.43}$$

Where δ_{jn} is the Kronecker delta function.

A.2) Optional Material for Section 2

We seek to prove compactness of K by the integral criterion, which states that an integral operator is *compact* if its domain is finite and the *kernel* is square-integrable:

$$\int_0^1\int_0^1 |k(t,s)|^2\,\mathrm{d}s\,\mathrm{d}t = \int_0^1\int_0^t \left(\frac{1}{\sqrt{t-s}}\right)^2 \mathrm{d}s\,\mathrm{d}t = \int_0^1\int_0^t \frac{1}{t-s}\,\mathrm{d}s\,\mathrm{d}t < \infty \tag{A.44}$$

Where by inspection we observe a singularity at the line $s = t$, which we handle by introducing K_ε with a small parameter $\varepsilon > 0$:

$$K_\varepsilon x(t) = \int_0^t \frac{1}{\sqrt{t-s+\varepsilon}} x(s)\,\mathrm{d}s \tag{A.45}$$

Such that the integral criterion becomes:

$$\int_0^1\int_0^1 |k_{\varepsilon(t,s)}|^2\,\mathrm{d}s\,\mathrm{d}t = \int_0^1\int_0^t \frac{1}{t-s+\varepsilon}\,\mathrm{d}s\,\mathrm{d}t < \infty \tag{A.46}$$

Using substitution $u = t - s + \varepsilon$ we solve the inner integral:

$$\int_0^t \frac{1}{t-s+\varepsilon}\,\mathrm{d}s = -\int_{t+\varepsilon}^{\varepsilon} \frac{1}{u}\,\mathrm{d}u = [\ln|u|]_\varepsilon^{t+\varepsilon} = \ln(t+\varepsilon) - \ln(\varepsilon) \tag{A.47}$$

Which is integrable and finite for $t \in [0,1]$ and $\varepsilon > 0$ by inspection.

As such, we now need to show that $K_\varepsilon \to K$ as $\varepsilon \to 0$. We do this by considering the difference between the two operators:

$$D_\varepsilon = K - K_\varepsilon = \int_0^t d_{\varepsilon(t,s)} x(s)\,\mathrm{d}s \tag{A.48}$$

Where $d_{\varepsilon(t,s)} = \frac{1}{\sqrt{t-s}} - \frac{1}{\sqrt{t-s+\varepsilon}}$ is the difference operator kernel.

We consider the following definition of the operator norm of the difference operator:

$$\|D_\varepsilon\|_{\text{op}} = \sup_{\|x\|_{L^2}=1} \|D_\varepsilon x\|_{L^2} \tag{A.49}$$

We thus seek to investigate the 2-norm of difference operator subject to $\|x\|_{L^2} = 1$, to which end we employ *Young's Convolution Inequality*:

$$\|f * g\|_r \leq \|f\|_p \cdot \|g\|_q \tag{A.50}$$

Where $1/p + 1/q = 1/r + 1$ and $1 \leq p, q, r \leq \infty$.

As such, with choices $r = p = 2, q = 1$ we find:

$$\|D_\varepsilon\|_{L^2} \leq \|d_\varepsilon(t,s)\|_{L^1} \cdot \|x\|_{L^2} = \|d_\varepsilon(t,s)\|_{L^1} \tag{A.51}$$

Which finally leads us to consider the L^1-norm of the difference operator kernel:

$$\|d_\varepsilon(t,s)\|_{L^1} = \int_0^t \left| \frac{1}{\sqrt{t-s}} - \frac{1}{\sqrt{t-s+\varepsilon}} \right| \mathrm{d}s \tag{A.52}$$

Which we again solve using substitution $u = t - s$:

$$\|d_\varepsilon(t,s)\|_{L^1} = \int_0^1 \left(\frac{1}{\sqrt{u}} - \frac{1}{\sqrt{u+\varepsilon}} \right) \mathrm{d}u = \left[2\sqrt{u} - 2\sqrt{u+\varepsilon} \right]_0^1 = 2 - 2\sqrt{1+\varepsilon}) + 2\sqrt{\varepsilon} \tag{A.53}$$

Combining the results above, we are able to bound the operator norm:

$$\|D_\varepsilon\|_{\text{op}} \leq \sup\left(2 - 2\sqrt{1+\varepsilon} + 2\sqrt{\varepsilon} \right) \tag{A.54}$$

Taking the limit $\varepsilon \to 0^+$ we find:

$$\|K - K_\varepsilon\|_{\text{op}} \to 0 \quad \text{as} \quad \varepsilon \to 0^+ \tag{A.55}$$

Since $K_\varepsilon \to K$ in the operator norm as $\varepsilon \to 0^+$ and K_ε is compact, we conclude that K is compact.

A.3) Large Figures for Section 1

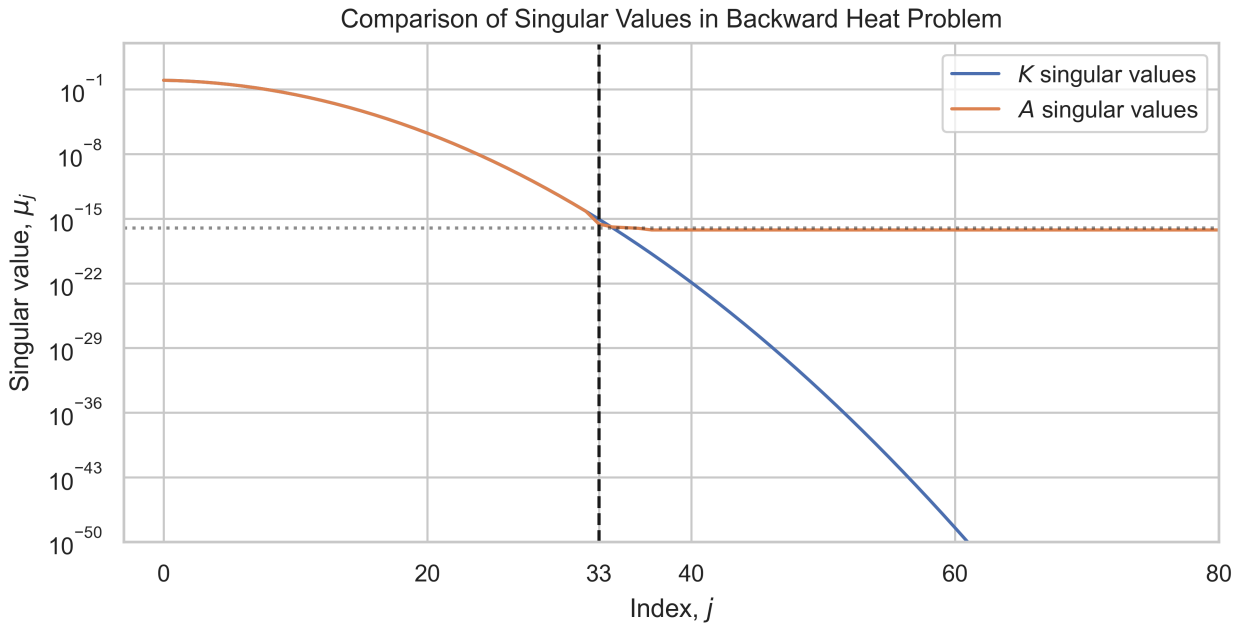


Figure 14: Singular values of the discretised operator A for $n = 128$ and $T = 0.03$ compared against the analytical expression for the singular values of K.

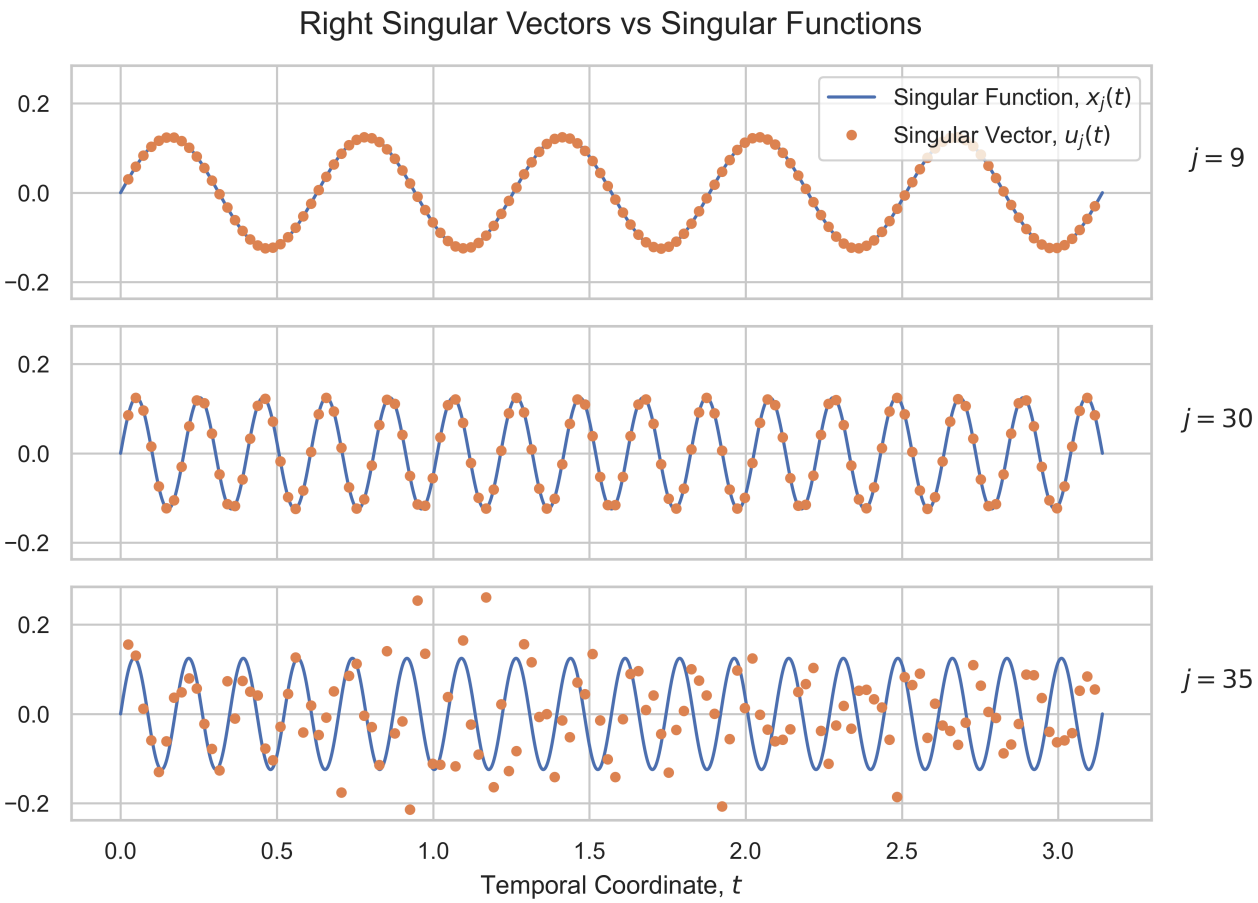


Figure 15: 3 select singular vectors and their associated singular functions for $n = 128$ and $T = 0.03$.

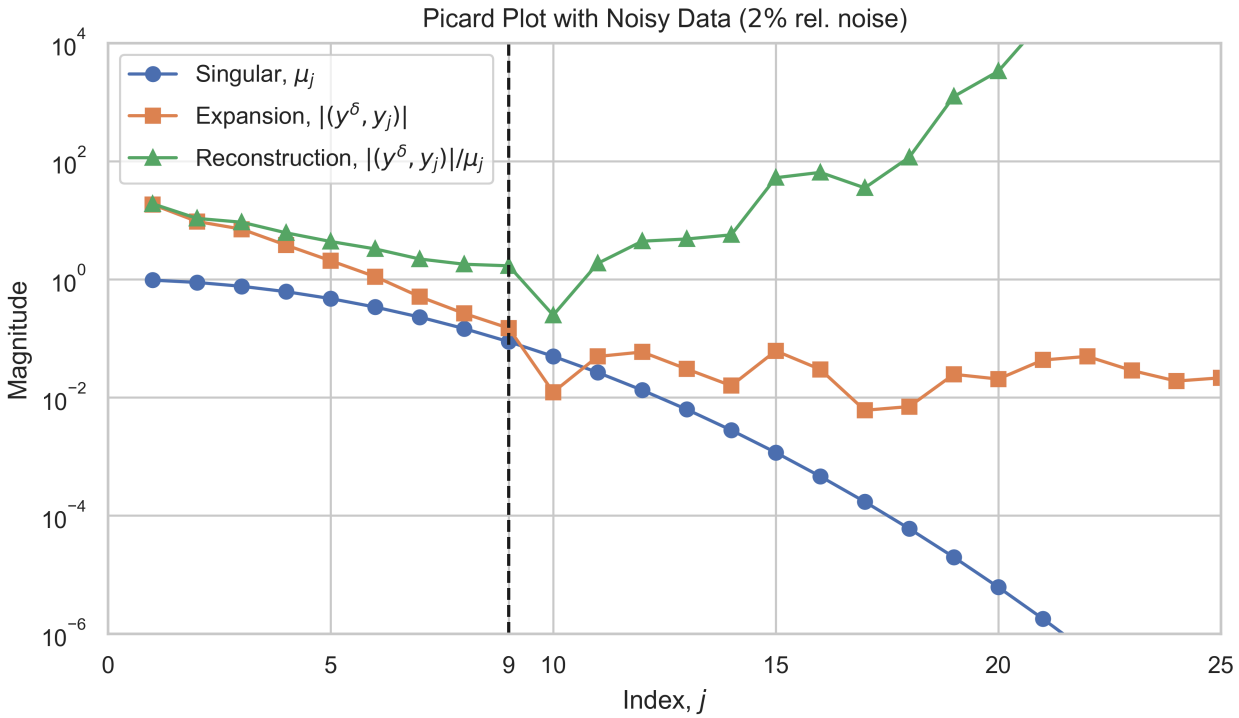


Figure 16: Picard plot of the noisy data y^δ with 2% relative noise.

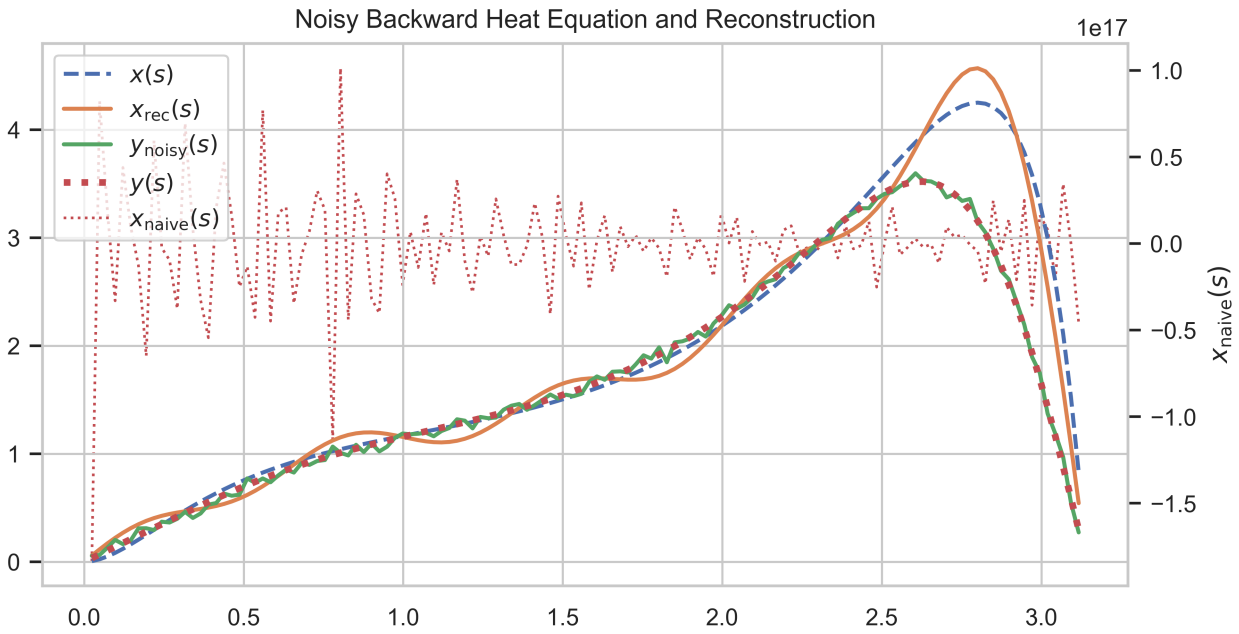


Figure 17: Reconstruction of x using truncated SVD with $j^\star = 9$ compared against the true x.

A.4) Large Figures for Section 2

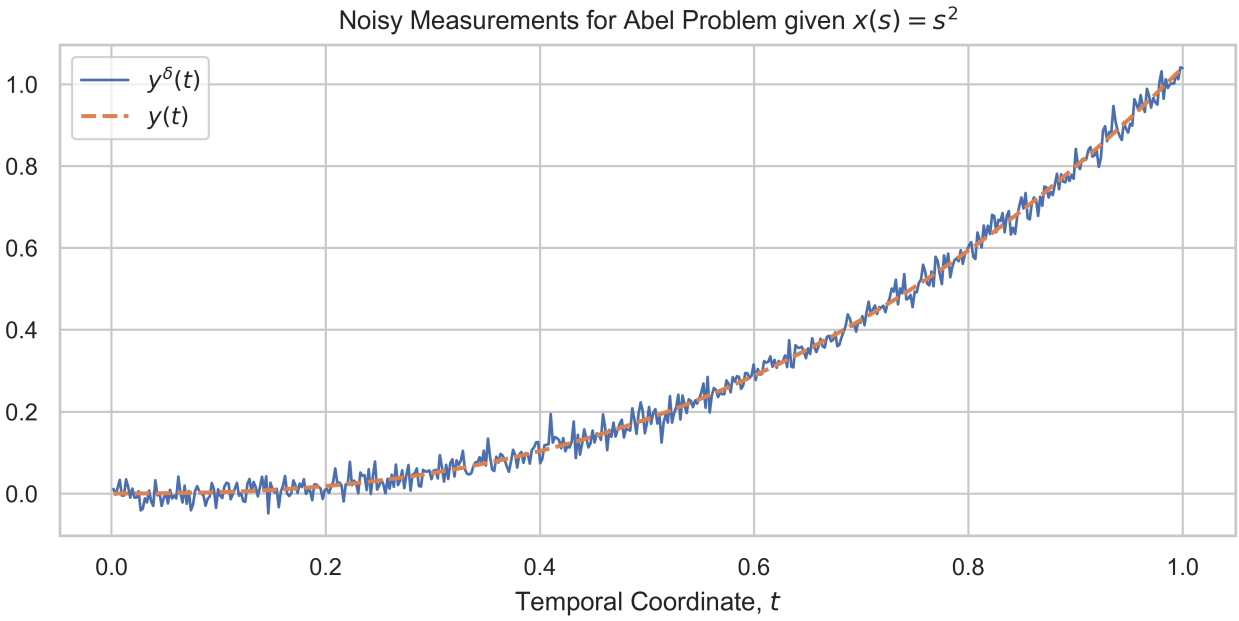


Figure 18: Noisy realisation of Abel problem with $x(s) = s^2$ perturbed by 5% relative white noise.

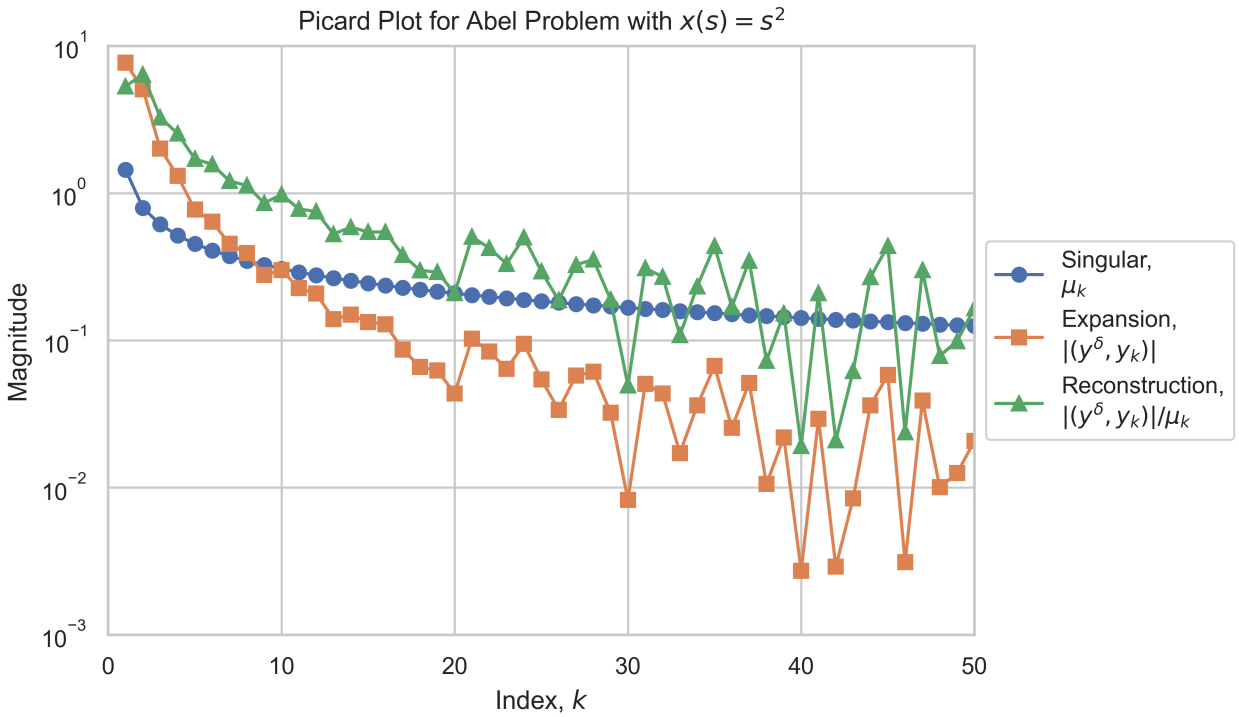


Figure 19: Picard plot of the noisy data y^δ with 5% relative noise.

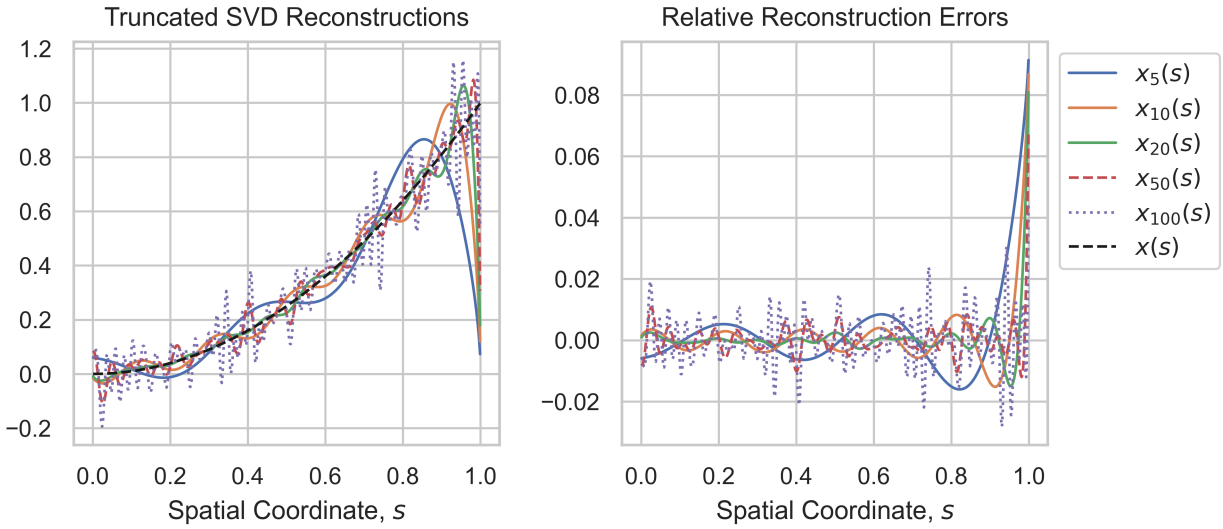


Figure 20: Reconstructions of $x(t)$ using truncated SVD with varying truncation levels k compared against the true $x(t)$.

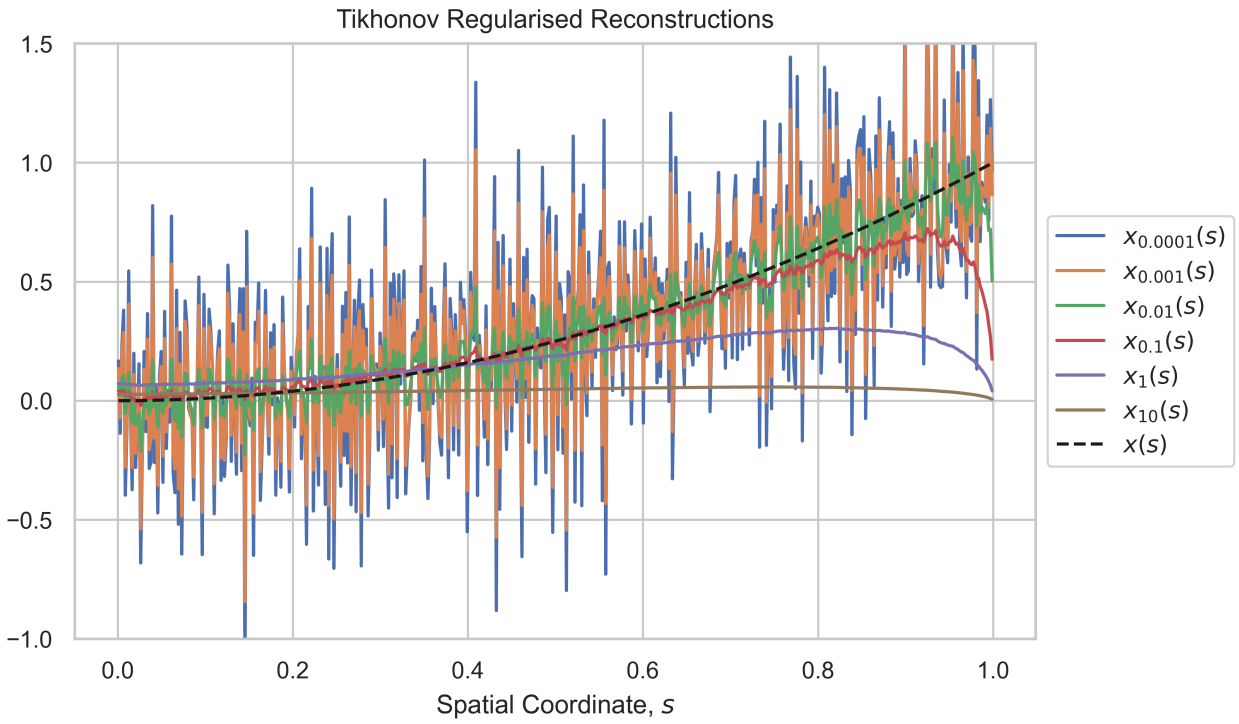


Figure 21: Reconstructions of $x(t)$ using Tikhonov regularisation with varying regularisation parameters α compared against the true $x(t)$.

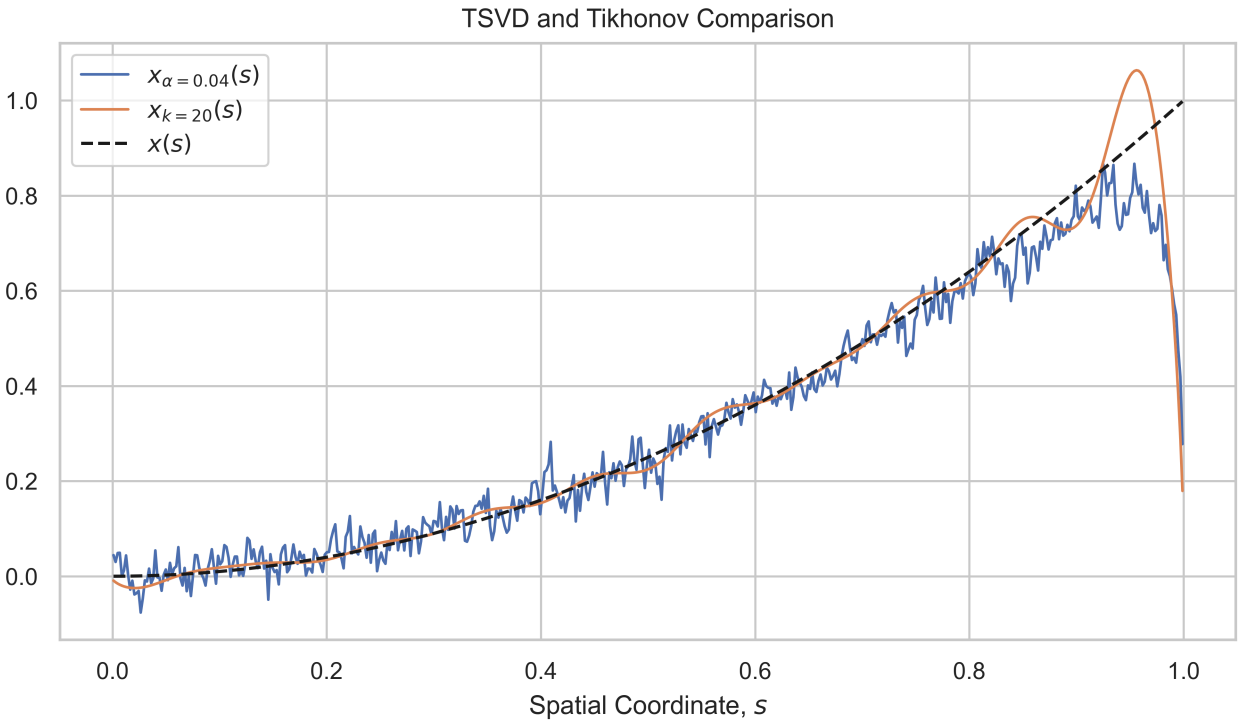


Figure 22: Comparison of TSVD reconstruction with $k = 20$ and Tikhonov reconstruction with $\alpha = 4 \times 10^{-2}$.

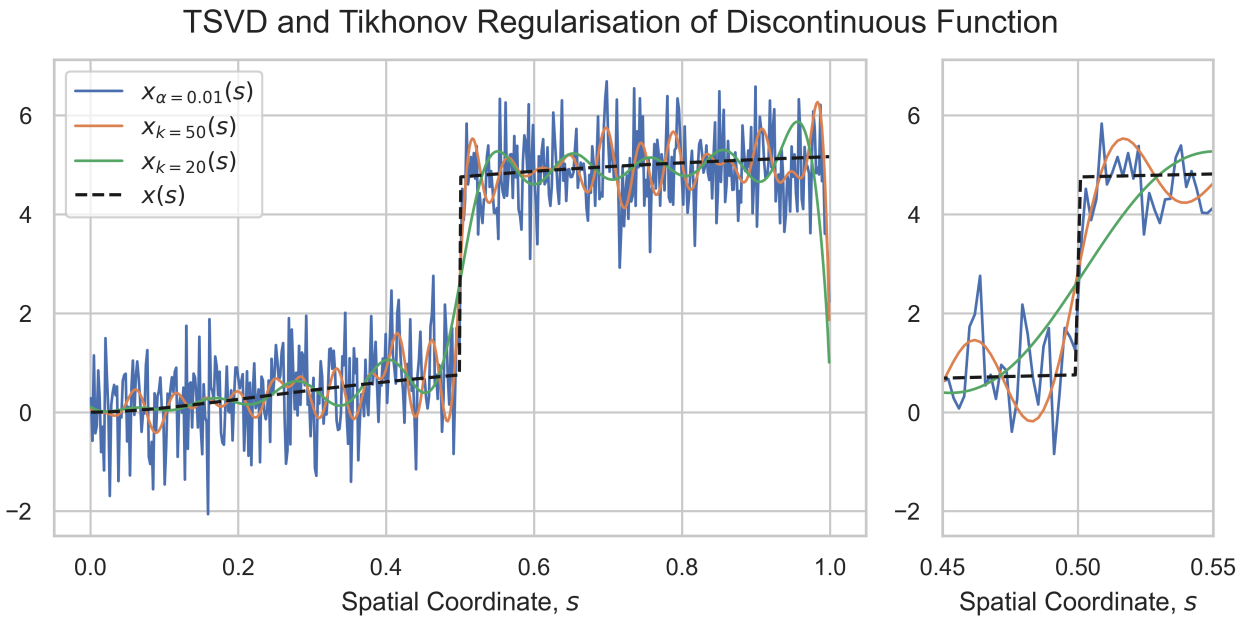


Figure 23: Comparison of TSVD reconstructions with $k \in \{20, 50\}$ and Tikhonov reconstruction with $\alpha = 10^{-2}$ for a discontinuous function.

A.5) Large Figures for Section 3

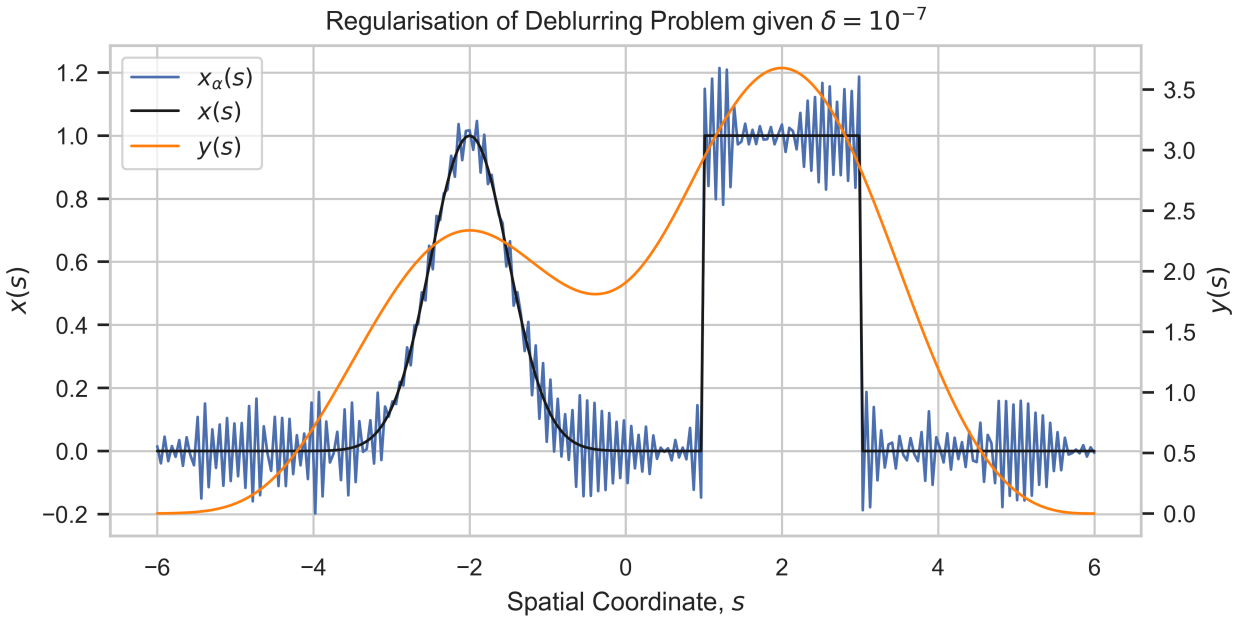


Figure 24: Validation of deblurring implementation with $\delta = 10^{-7}$ and $\alpha = \delta^3$.

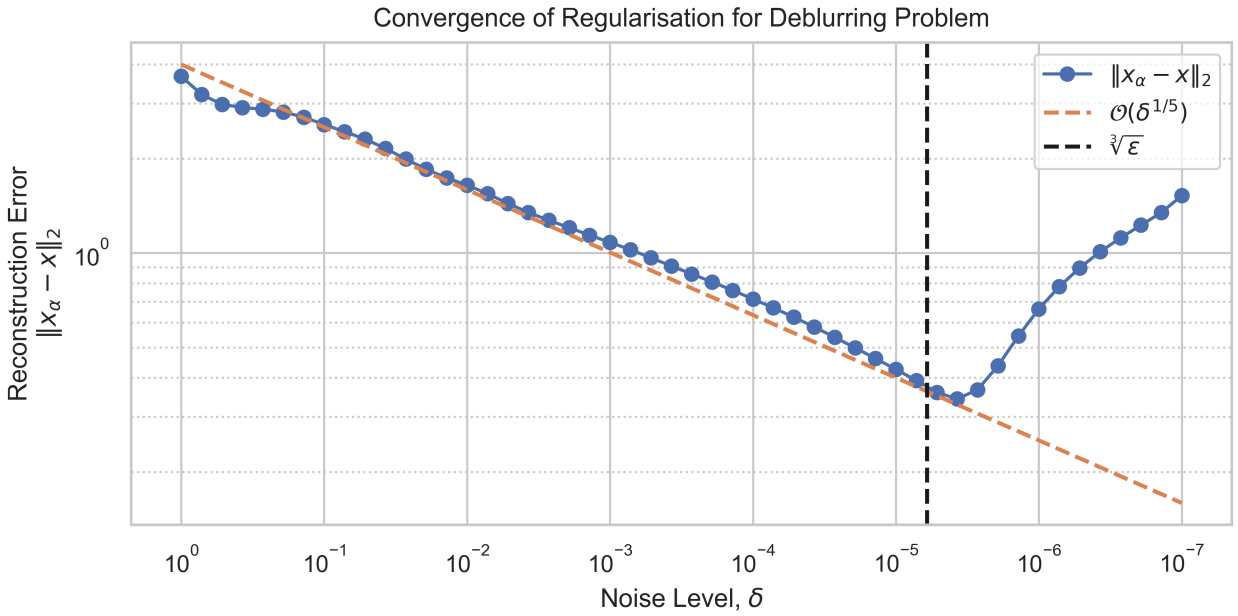


Figure 25: Convergence of regularised solution x_α to true solution x as noise level $\delta \to 0$ with $\alpha = \delta^3$.

A.6) Code for Section 1.b

Python

```
T = 0.03
n = 128

k = np.arange(1, n + 1)
t = k * np.pi / (n + 1)
s = k * np.pi / (n + 1)

# We construct A matrix using spectral method
# Alternative would be to use a quadrature rule

# Create left and right singular vectors sampled at t_i, s_i
j = np.arange(1, n + 1)
U = np.sqrt(2 / (n + 1)) * np.sin(np.einsum("i,j->ij", t, k))
V = np.sqrt(2 / (n + 1)) * np.sin(np.einsum("i,j->ij", s, k))
assert np.allclose(U.T @ U, np.eye(n)), "V is not orthogonal"
assert np.allclose(V.T @ V, np.eye(n)), "V is not orthogonal"

# Create diagonal matrix S of singular values
S = np.diag(np.exp(-(j**2) * T))

# Now we can construct A = U D V^*
A = U @ S @ V.T

# Now we can compare as requested in assignment
# This is almost cheating since we constructed A from its SVD
# But it would yield the same results if constructed using a sufficiently
# accurate quadrature rule
U_svd, S_svd, Vh_svd = np.linalg.svd(A)
plt.figure(figsize=(PAPER_WIDTH_IN, 2.0))
plt.plot(np.diagonal(S), label="$K$ singular values")
plt.plot(S_svd, label="$A$ singular values")
plt.axvline(33, color="k", ls="--")
```

```
plt.axhline(1e-16, color="k", ls=":", alpha=0.5)
plt.xticks(list(plt.xticks()[0]) + [33])
plt.semilogy()
plt.title("Comparison of Singular Values in Backward Heat Problem")
plt.xlabel("Index, $j$")
plt.ylabel("Singular value, $μ_j$")
plt.legend()
plt.ylim(1e-50, 1e4)
plt.xlim(0 - 3, 80)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_b_singular_values.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 3.5)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_b_singular_values_large.png", dpi=600,
bbox_inches="tight")


# Singular vectors vs functions
plt.figure()


def singular_vector_func(j, x):
    return np.sqrt(2 / np.pi) * np.sin(j * np.pi * x)


t_i_fine = np.linspace(0, np.pi, 2500)
fig, axes = plt.subplots(3, 1, figsize=(PAPER_WIDTH_IN, 2.7), sharex=True,
sharey=True)
for i, (ax, j_k) in enumerate(zip(axes, [9, 30, 35])):
    u_calculated = singular_vector_func(j_k + 1, t_i_fine / np.pi)
    u_calculated *= np.sqrt(t[1] - t[0])  # Normalization factor
    ax.plot(t_i_fine, u_calculated, label=f"Singular Function, $x_j(t)$")
    ax.plot(t, U_svd[:, j_k], "o", ms=3, label="Singular Vector, $u_j(t)$")
    ax.set_title(f"$j={j_k}$", x=1.06, y=0.5)

    if i == 0:
        ax.legend()

    if i == 2:
        ax.set_xlabel("Temporal Coordinate, $t$")

fig.suptitle("Right Singular Vectors vs Singular Functions", y=0.92, fontsize=10)
fig.tight_layout()
fig.subplots_adjust(hspace=0.08)
fig.savefig(OUTPUT_DIR / "1_b_singular_vectors.png", dpi=600, bbox_inches="tight")
display(fig)

fig.set_size_inches(PAPER_WIDTH_IN, 5)
fig.suptitle(fig._suptitle.get_text(), y=0.97, fontsize=12)
```

```
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "1_b_singular_vectors_large.png", dpi=600,
bbox_inches="tight")
```

A.7) Code for Section 1.c

Python

```
np.random.seed(42)


def x_func(s):
    return s * np.sin(s) * np.exp((s - np.pi / 2) ** 2)


x = x_func(s)
y = A @ x
unscaled_noise = np.random.randn(*y.shape)
unscaled_noise_level = np.linalg.norm(unscaled_noise)
noise_coeff = 0.02 * np.linalg.norm(y) / unscaled_noise_level
noise = noise_coeff * unscaled_noise
y_noisy = y + noise
x_naive = np.linalg.inv(A) @ y_noisy

# Prepare for Picard plot
singular_values = np.diagonal(S)
expansion_coefficients = np.abs(U.T @ y_noisy)
reconstruction_coefficients = expansion_coefficients / singular_values
plt.figure(figsize=(PAPER_WIDTH_IN, 2.3))
plt.plot(j, singular_values, marker="o", ls="-", ms=5, label="Singular, $μ_j$")
plt.plot(
    j,
    expansion_coefficients,
    marker="s",
    ls="-",
    ms=5,
    label="Expansion, $|(y^δ, y_j)|$",
)
plt.plot(
    j,
    reconstruction_coefficients,
    marker="^",
    ls="-",
    ms=5,
    label="Reconstruction, $|(y^δ, y_j)|/μ_j$",
)
plt.semilogy()
plt.legend()
plt.xlim(0, 25)
plt.ylim(1e-6, 1e4)
plt.axvline(9, color="k", ls="--")
plt.xticks(list(plt.xticks()[0]) + [9])
plt.title("Picard Plot with Noisy Data ($2\\%$ rel. noise)")
plt.xlabel("Index, $j$")
```

```python
plt.ylabel("Magnitude")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c_picard_plot.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 4)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c_picard_plot_large.png", dpi=600, bbox_inches="tight")

# Compute reconstruction
j_cutoff = 9  # Inclusive
S_truncated = S[:j_cutoff, :j_cutoff]
U_truncated = U[:, :j_cutoff]
V_truncated = V.T[:, :j_cutoff]
K_inv_truncated = V_truncated @ np.linalg.inv(S_truncated) @ U_truncated.T
x_reconstructed = K_inv_truncated @ y_noisy

plt.figure(figsize=(PAPER_WIDTH_IN, 2.0))
plt.plot(s, x_func(s), label="$x(s)$", ls="--", lw=1.5)
plt.plot(s, x_reconstructed, label="$x_\\text{rec}(s)$", lw=1.5)
plt.plot(s, y_noisy, label="$y_\\text{noisy}(s)$", lw=1.5)
plt.plot(s, y, label="$y(s)$", ls=":", lw=2.5)
plt.legend()

# Create new axis for naive solution
ax_main = plt.gca()
ax_noisy = plt.twinx()
ax_noisy.plot(s, x_naive, label="$x_\\text{naive}(s)$", color="C3", ls=":", lw=1)
ax_noisy.grid(False)
ax_main.set_zorder(1)
ax_main.patch.set_visible(False)
ax_noisy.set_ylabel("$x_\\text{naive}(s)$")

# Merge legends
handles1, labels1 = ax_main.get_legend_handles_labels()
handles2, labels2 = ax_noisy.get_legend_handles_labels()
ax_main.legend(handles1 + handles2, labels1 + labels2, loc="upper left")

plt.xlabel("Spatial Coordinate, $s$")
plt.title("Noisy Backward Heat Equation and Reconstruction")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c_reconstruction.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 3.5)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c_reconstruction_large.png", dpi=600,
bbox_inches="tight")
```

A.8) Code for Section 2.a

```python
np.random.seed(42)
```
Python

```python

n = 512
a = 0
b = 1
h = (b - a) / n
print(np.sqrt(2 / h) * h)
s = a + (np.arange(n) + 0.5) * h
t = s + h / 2  # Shift t values to prevent singularity at t=s

s_mesh, t_mesh = np.meshgrid(s, t)

# Create A matrix
A = np.zeros((n, n))
indices = s_mesh < t_mesh  # Select according to integral
A[indices] = h / np.sqrt((t_mesh[indices] - s_mesh[indices]))


def x_1_func(s):
    return s**2


x_true = x_1_func(s)
y_true = A @ x_true

# Add noise
unscaled_noise = np.random.randn(*y_true.shape)
unscaled_noise_level = np.linalg.norm(unscaled_noise)
noise_coeff = 0.05 * np.linalg.norm(y_true) / unscaled_noise_level
noise = noise_coeff * unscaled_noise
y_noisy = y_true + noise

# Plot noisy measurements
plt.figure(figsize=(PAPER_WIDTH_IN, 2.2))
plt.plot(t, y_noisy, label="$y^δ(t)$", lw=1)
plt.plot(t, y_true, label="$y(t)$", lw=1.5, ls="--")
plt.title("Noisy Measurements for Abel Problem given $x(s) = s^2$")
plt.xlabel("Temporal Coordinate, $t$")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_a.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 3.5)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_a_large.png", dpi=600, bbox_inches="tight")
```

A.9) Code for Section 2.b

```python
# Compute naive solution
x_naive = np.linalg.inv(A) @ y_noisy

# Perform SVD
```

```
j = np.arange(1, n + 1)
U, S, Vh = np.linalg.svd(A)

# Prepare Picard
expansion_coefficients = np.abs(U.T @ y_noisy)
reconstruction_coefficients = expansion_coefficients / S
# Perform Picard plot
plt.figure(figsize=(PAPER_WIDTH_IN, 2.1))
plt.title("Picard Plot for Abel Problem with $x(s) = s^2$")
plt.plot(j, S, marker="o", ls="-", ms=5, label="Singular,\n$μ_k$")
plt.plot(
    j,
    expansion_coefficients,
    marker="s",
    ls="-",
    ms=5,
    label="Expansion,\n$|(y^δ, y_k)|$",
)
plt.plot(
    j,
    reconstruction_coefficients,
    marker="^",
    ls="-",
    ms=5,
    label="Reconstruction,\n$|(y^δ, y_k)|/μ_k$",
)
plt.xlim(0, 50)
plt.ylim(1e-3, 1e1)
plt.xlabel("Index, $k$")
plt.ylabel("Magnitude")
plt.semilogy()
plt.legend(loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_b_picard.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 4)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_b_picard_large.png", dpi=600, bbox_inches="tight")

# Reconstruction plots
fig, (ax_real, ax_error) = plt.subplots(
    1, 2, figsize=(PAPER_WIDTH_IN, 2.0), sharex=True
)

# ks = [5, 10, 25, 50, 100, "naive"]
ks = [5, 10, 20, 50, 100]
plot_kws_config = {
    50: {"ls": "--"},
    100: {"ls": ":"},
```

```
}
for k in ks:
    if k == "naive":
        x_reconstructed = x_naive
        show_error = False
    else:
        # To TSVD
        U_truncated = U[:, :k]
        S_truncated = np.diag(S)[:k, :k]
        V_truncated = Vh.T[:, :k]
        A_inv_truncated = V_truncated @ np.linalg.inv(S_truncated) @ U_truncated.T
        x_reconstructed = A_inv_truncated @ y_noisy
        show_error = True

    plot_kws = plot_kws_config.get(k, {})
    plot_kws.setdefault("lw", 1)
    plot_kws.setdefault("ls", "-")
    real_plot = ax_real.plot(s, x_reconstructed, label=f"$x_{{{k}}}(s)$",
    **plot_kws)

    if show_error:
        residual = x_true - x_reconstructed
        relative_error = (x_true - x_reconstructed) / np.linalg.norm(x_true)
        ax_error.plot(s, relative_error, color=real_plot[0].get_color(), **plot_kws)

ax_real.plot(s, x_true, label="$x(s)$", lw=1, ls="--", color="k")
ax_real.set_xlabel("Spatial Coordinate, $s$")
ax_error.set_xlabel("Spatial Coordinate, $s$")
ax_real.set_title("Truncated SVD Reconstructions")
ax_error.set_title("Relative Reconstruction Errors")

# Legend stuff
# Collect all handles and labels from both axes
handles_real, labels_real = ax_real.get_legend_handles_labels()
handles_error, labels_error = ax_error.get_legend_handles_labels()

# Place combined legend on ax_error, to the right
ax_error.legend(
    handles_real + handles_error,
    labels_real + labels_error,
    loc="upper left",
    bbox_to_anchor=(1, 1),
)

fig.tight_layout()
fig.savefig(OUTPUT_DIR / "2_b_reconstructions.png", dpi=600, bbox_inches="tight")
display(fig)

fig.set_size_inches(PAPER_WIDTH_IN, 3)
fig.tight_layout()
```

```
fig.savefig(OUTPUT_DIR / "2_b_reconstructions_large.png", dpi=600, bbox_inches="tight")
```

A.10) Code for Section 2.c

Python

```
alphas = np.geomspace(1e-4, 10, 10)
alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 10]

plt.figure(figsize=(PAPER_WIDTH_IN, 2.2))
for alpha in alphas:
    x_tikhonov = np.linalg.inv(A.T @ A + alpha * np.eye(n)) @ A.T @ y_noisy
    plt.plot(s, x_tikhonov, label=f"$x_{{{alpha}}}(s)$")

# plt.plot(s, x_naive, label='$x_{\\mathrm{naive}}(s)$', ls=":")
plt.plot(s, x_true, label="$x(s)$", ls="--", color="k")
plt.legend(loc="center left", bbox_to_anchor=(1, 0.5))
plt.title("Tikhonov Regularised Reconstructions")
plt.xlabel("Spatial Coordinate, $s$")
plt.ylim(-1, 1.5)
plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "2_c_tikhonov_reconstructions.png", dpi=600, bbox_inches="tight"
)
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 4)
plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "2_c_tikhonov_reconstructions_large.png", dpi=600,
    bbox_inches="tight"
)
# Best TSVD
k = 20
U, S, Vh = np.linalg.svd(A)
U_truncated = U[:, :k]
S_truncated = np.diag(S)[:k, :k]
V_truncated = Vh.T[:, :k]
A_inv_truncated = V_truncated @ np.linalg.inv(S_truncated) @ U_truncated.T
x_best_tsvd = A_inv_truncated @ y_noisy

# Best Tikhonov
alpha = 4e-2
x_best_tikhonov = np.linalg.inv(A.T @ A + alpha * np.eye(n)) @ A.T @ y_noisy

plt.figure(figsize=(PAPER_WIDTH_IN, 2.1))
plt.plot(s, x_best_tikhonov, label=f"$x_{{α={alpha}}}(s)$", lw=1)
plt.plot(s, x_best_tsvd, label=f"$x_{{k={k}}}(s)$", lw=1)
plt.plot(s, x_true, label="$x(s)$", ls="--", color="k")
plt.legend()
plt.title("TSVD and Tikhonov Comparison")
plt.xlabel("Spatial Coordinate, $s$")
plt.tight_layout()
```

```
plt.savefig(OUTPUT_DIR / "2_c_tsvd_vs_tikhonov.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 4)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_c_tsvd_vs_tikhonov_large.png", dpi=600,
bbox_inches="tight")
```

A.11) Code for Section 2.d

Python

```

def heaviside_step(x):
    return 0.5 * (np.sign(x) + 1)


def x_func_discontinuous(s):
    return x_func(s) + 4 * heaviside_step(s - 1 / 2)


x_true = x_func_discontinuous(s)
y_true = A @ x_true

# Add noise
np.random.seed(1234)
unscaled_noise = np.random.randn(*y_true.shape)
unscaled_noise_level = np.linalg.norm(unscaled_noise)
noise_coeff = 0.05 * np.linalg.norm(y_true) / unscaled_noise_level
noise = noise_coeff * unscaled_noise
y_noisy = y_true + noise

# Best TSVD
k_best = 50
U, S, Vh = np.linalg.svd(A)
U_truncated = U[:, :k_best]
S_truncated = np.diag(S)[:k_best, :k_best]
V_truncated = Vh.T[:, :k_best]
A_inv_truncated = V_truncated @ np.linalg.inv(S_truncated) @ U_truncated.T
x_best_tsvd = A_inv_truncated @ y_noisy

# Slow TSVD
k_slow = 20
U, S, Vh = np.linalg.svd(A)
U_truncated = U[:, :k_slow]
S_truncated = np.diag(S)[:k_slow, :k_slow]
V_truncated = Vh.T[:, :k_slow]
A_inv_truncated = V_truncated @ np.linalg.inv(S_truncated) @ U_truncated.T
x_slow_tsvd = A_inv_truncated @ y_noisy

# Best Tikhonov
alpha = 1e-2
x_best_tikhonov = np.linalg.inv(A.T @ A + alpha * np.eye(n)) @ A.T @ y_noisy
```

```
fig, (ax_main, ax_zoom) = plt.subplots(
    1, 2, figsize=(PAPER_WIDTH_IN, 2.3), gridspec_kw={"width_ratios": [3, 1]}
)
ax_main.plot(s, x_best_tikhonov, label=f"$x_{{α={alpha}}}(s)$", lw=1)
ax_zoom.plot(s, x_best_tikhonov, label=f"$x_{{α={alpha}}}(s)$", lw=1)
ax_main.plot(s, x_best_tsvd, label=f"$x_{{k={k_best}}}(s)$", lw=1)
ax_zoom.plot(s, x_best_tsvd, label=f"$x_{{k={k_best}}}(s)$", lw=1)
ax_main.plot(s, x_slow_tsvd, label=f"$x_{{k={k_slow}}}(s)$", lw=1)
ax_zoom.plot(s, x_slow_tsvd, label=f"$x_{{k={k_slow}}}(s)$", lw=1)
ax_main.plot(s, x_true, label="$x(s)$", ls="--", color="k")
ax_zoom.plot(s, x_true, label="$x(s)$", ls="--", color="k")
ax_main.legend()
fig.suptitle(
    "TSVD and Tikhonov Regularisation of Discontinuous Function", y=0.90, fontsize=10
)
ax_main.set_xlabel("Spatial Coordinate, $s$")
ax_zoom.set_xlabel("Spatial Coordinate, $s$")
ax_zoom.set_xlim(0.45, 0.55)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "2_d_discontinuous.png", dpi=600, bbox_inches="tight")
display(fig)

fig.set_size_inches(PAPER_WIDTH_IN, 3.5)
fig.suptitle(fig._suptitle.get_text(), y=0.95, fontsize=12)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "2_d_discontinuous_large.png", dpi=600, bbox_inches="tight")
```

A.12) Code for Section 3.b

Python

```
# Deblurring problem
n = 2**8
h = 12 / n
n4 = n // 4
delta = 1e-7
alpha = delta**3

r1 = np.zeros(n)

# Cosine values for the formula
c = np.cos(np.arange(-1, n4 + 1) * 4 * np.pi / n)

# Fill in the first n/4 entries
r1[:n4] = h + 9 / (h * np.pi**2) * (2 * c[1 : n4 + 1] - c[:n4] - c[2 : n4 + 2])

# Middle entry
r1[n4] = h / 2 + 9 / (h * np.pi**2) * (np.cos(4 * np.pi / n) - 1)

A = toeplitz(r1)


# Set up true image
```

```
def tophat_func(s, width, center):
    return heaviside_step(s - (center - width / 2)) - heaviside_step(
        s - (center + width / 2)
    )


def x_deblur_func(s):
    return np.exp(-((s + 2) ** 2) / 0.5) + tophat_func(s, 2, 2)


s = np.linspace(-6, 6, n)

x_true = x_deblur_func(s)
y_true = A @ x_true

# Add noise
np.random.seed(42)
unscaled_noise = np.random.randn(*y_true.shape)
unscaled_noise_level = np.linalg.norm(unscaled_noise)
noise_coeff = delta * np.linalg.norm(y_true) / unscaled_noise_level
noise = noise_coeff * unscaled_noise
y_noisy = y_true + noise

# Compute regularisation
U, S, Vh = np.linalg.svd(A)

Y_basis = U
X_basis = Vh.T
mu = S
y_yj_ip = Y_basis.T @ y_noisy
filter_factors = mu**2 / (mu**3 + alpha)
coefficients = filter_factors * y_yj_ip
x_deblurred = X_basis @ coefficients

# Naive solution
x_true_naive = np.linalg.inv(A) @ y_true
x_naive = np.linalg.inv(A) @ y_noisy

fig, ax_x = plt.subplots(figsize=(PAPER_WIDTH_IN, 2.0))
ax_y = ax_x.twinx()
# ax_x.plot(s, x_true_naive, label='$x_{\\mathrm{naive}}(s)$', lw=1)
# ax_x.plot(s, x_naive, label='$x_{\\mathrm{naive}}^δ(s)$', lw=1)
ax_x.plot(s, x_deblurred, label="$x_α(s)$", lw=1)
ax_x.plot(s, x_true, label="$x(s)$", lw=1, c="k")
ax_y.plot(s, y_true, label="$y(s)$", lw=1, c="tab:orange")

ax_y.grid(False)

# Legend
handles_x, labels_x = ax_x.get_legend_handles_labels()
```

```
handles_y, labels_y = ax_y.get_legend_handles_labels()
ax_x.legend(handles_x + handles_y, labels_x + labels_y, loc="upper left")
ax_x.set_xlabel("Spatial Coordinate, $s$")
ax_x.set_ylabel("$x(s)$")
ax_y.set_ylabel("$y(s)$")

ax_x.set_title(
    f"Regularisation of Deblurring Problem given $δ={sci_notation(delta, 0)}$"
)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "3_b_validation.png", dpi=600, bbox_inches="tight")
display(fig)

fig.set_size_inches(PAPER_WIDTH_IN, 3.5)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "3_b_validation_large.png", dpi=600, bbox_inches="tight")
# Investigate convergence
deltas = np.geomspace(1.0, 1e-7, 50)

machine_prec = np.finfo(float).eps
machine_prec_transformed = np.cbrt(machine_prec)

convergence_data = []
for delta in deltas:
    alpha = delta**3

    # Compute regularisation
    y_yj_ip = Y_basis.T @ y_noisy
    filter_factors = mu**2 / (mu**3 + alpha)
    coefficients = filter_factors * y_yj_ip
    x_deblurred = X_basis @ coefficients

    error_norm = np.linalg.norm(x_deblurred - x_true, ord=2)
    convergence_data.append({"delta": delta, "alpha": alpha, "error_norm":
    error_norm})


df = pd.DataFrame(convergence_data)

fig, ax = plt.subplots(figsize=(PAPER_WIDTH_IN, 2.0))
ax.xaxis.set_inverted(True)
plt.plot(df["delta"], df["error_norm"], marker="o", ls="-", label="$\\|x_α - x\\|
_2$")
plt.plot(
    df["delta"],
    4 * df["delta"] ** (1 / 5),
    ls="--",
    lw=1.5,
    label="$\\mathcal{O}(δ^{1/5})$",
)
plt.axvline(
```

```
    machine_prec_transformed, color="k", ls="--", lw=1.5, label="$\\sqrt[3]{ε}$"
)
plt.loglog()
plt.grid(axis="y", which="minor", ls=":")
plt.legend()
plt.title("Convergence of Regularisation for Deblurring Problem")
plt.xlabel("Noise Level, $δ$")
plt.ylabel("Reconstruction Error\n$\\|x_α - x\\|_2$")
plt.tight_layout()
fig.savefig(OUTPUT_DIR / "3_b_convergence.png", dpi=600, bbox_inches="tight")
display(fig)

fig.set_size_inches(PAPER_WIDTH_IN, 3.5)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "3_b_convergence_large.png", dpi=600, bbox_inches="tight")
```

Bibliography

[1] A. Kirsch, *An Introduction to the Mathematical Theory of Inverse Problems*, vol. 120. in Applied Mathematical Sciences, vol. 120. Cham: Springer International Publishing, 2021. doi: 10.1007/978-3-030-63343-1.

[2] P. C. Hansen, *Discrete Inverse Problems: Insight and Algorithms*, no. 7. in Fundamentals of Algorithms. Philadelphia: Society for Industrial and Applied Mathematics, 2010.